Tutoriel Python pour débutants

Préparation aux TP 1 et 2

Analyse de données pour physiciens

Licence de Physique — L2 / L3

Durée estimée : 7 à 9 heures — à faire avant le premier TP

Prérequis : aucun en programmation

Document pédagogique — Version 1.0

Table des matières

1 Avant de commencer **2**
1.1 Installation . . . 2
1.2 Vérification . . . 2
1.3 La règle d'or du débutant . . . 2

2 Comprendre et maîtriser les `import` **2**
2.1 Pourquoi `import` ? . . . 3
2.2 Les trois formes d'import . . . 3
2.3 La forme à éviter absolument : `from x import *` . . . 3
2.4 Le bloc d'import standard du cours . . . 4
2.5 Comment ça se passe dans un notebook . . . 4
2.6 Erreurs classiques et leur correction . . . 4
2.7 `pip install` vs `import` . . . 5
2.8 Bonnes pratiques à adopter dès maintenant . . . 5
2.9 Exercices . . . 5

3 Variables, types et opérations **6**
3.1 Variables et types . . . 6
3.2 Opérations arithmétiques . . . 6
3.3 Les f-strings, ton meilleur ami . . . 6

4 Listes et structures de contrôle **7**
4.1 Listes . . . 7
4.2 Boucles `for` . . . 7
4.3 Conditions `if / else` . . . 7

5 NumPy — le cœur du calcul scientifique **8**
5.1 Créer un tableau . . . 8
5.2 Opérations vectorisées (le super-pouvoir) . . . 8

6 Matplotlib — visualiser **9**
6.1 Figure minimale . . . 9
6.2 Histogramme . . . 9
6.3 Barres d'erreur (crucial pour les TP) . . . 9
6.4 Sous-figures (pour TP 1) . . . 9

7 Pandas — lire des fichiers de données **10**
7.1 DataFrame et CSV . . . 10
7.2 Accéder à une colonne . . . 10
7.3 Écrire un CSV . . . 10

8 Nombres aléatoires **10**

9 SciPy — distributions et ajustements **11**
9.1 Loi normale et ajustement . . . 11
9.2 QQ-plot (test de normalité, utilisé au TP 1) . . . 11
9.3 Ajustement linéaire pondéré (aperçu TP 3) . . . 11

10 Assemblage — tu es prêt pour les TP **12**
10.1 Squelette complet du TP 1 . . . 12
10.2 Cœur du TP 2 (propagation et Monte Carlo) . . . 12

11 Checklist avant le TP **13**

12 Dépannage — les blocages les plus fréquents **14**

13 Pour aller plus loin (optionnel) **14**

1 Avant de commencer

1.1 Installation

Objectifs de la section

À la fin de cette section, tu auras un environnement Python fonctionnel avec toutes les bibliothèques nécessaires aux TP 1 et 2.

Option A — Anaconda (recommandé pour débuter).

1. Télécharge Anaconda sur `https://www.anaconda.com/download`.
2. Installe-le avec les options par défaut.
3. Lance **Anaconda Navigator** → **JupyterLab** → **Launch**.
4. Crée un dossier `TP_analyse_donnees/` sur ton bureau et navigue dedans.

Option B — Google Colab (aucune installation).

1. Va sur `https://colab.research.google.com`.
2. Clique sur **Nouveau notebook**. Tout est prêt.

1.2 Vérification

Dans une cellule Jupyter, tape et exécute avec `Shift + Entrée` :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
print("NumPy", np.__version__)
print("Pandas", pd.__version__)
print("SciPy", scipy.__version__)
```

Si tu vois trois numéros de version, tout est en ordre. Sinon, reviens à l'installation.

1.3 La règle d'or du débutant

Rappels utiles

- Une cellule = un petit bout de code.
- `Shift + Entrée` pour exécuter.
- En cas d'erreur, **lis la dernière ligne du message** : Python y dit exactement ce qui ne va pas et à quelle ligne.
- En cas de comportement bizarre : **Kernel** → **Restart & Run All**.

2 Comprendre et maîtriser les `import`

Objectifs de la section

Comprendre ce qu'est un module Python, maîtriser les trois formes d'import, connaître les conventions et éviter les pièges classiques.

2.1 Pourquoi `import` ?

Python de base ne contient que peu de fonctions (`print`, `len`, `sum`...). Tout le reste — tableaux NumPy, lecture CSV, figures Matplotlib — vit dans des **modules** ou **bibliothèques** externes. Pour les utiliser, il faut les **importer**.

Analogie

`import` est comme sortir un livre de la bibliothèque. Tant que tu ne l'as pas emprunté, tu ne peux pas le lire. Une fois importé, tu peux le consulter dans toutes les cellules suivantes du notebook.

2.2 Les trois formes d'import

Forme 1 — Import simple.

```
import math
print(math.pi)          # 3.141592653589793
print(math.sqrt(4))     # 2.0
```

Tu importes **le module entier**. Toute fonction doit être préfixée : `math.sqrt`. Explicite et sûr.

Forme 2 — Import avec alias.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

x = np.array([1, 2, 3])
```

`as np` renomme le module **uniquement dans ton script**. C'est la convention universelle : tous les tutoriels, Stack Overflow et codes professionnels utilisent `np`, `pd`, `plt`. **Adopte-la sans réfléchir.**

Forme 3 — Import ciblé d'une fonction.

```
from scipy.stats import norm, probplot
from math import sqrt, pi

print(sqrt(4))          # 2.0
mu, sigma = norm.fit([1.0, 2.0, 3.0])
```

Pas de préfixe. Pratique quand tu utilises souvent la même fonction ; risqué si tu en importes trente de cinq modules différents.

2.3 La forme à éviter absolument : `from x import *`

Piège classique

```
from math import *      # NE PAS FAIRE
from numpy import *     # PIRE ENCORE
```

Pourquoi c'est une mauvaise idée :

- Tu pollues ton espace de noms avec des centaines de noms inconnus.
- Tu ne sais plus d'où vient `sqrt` (`math` ? `numpy` ? `scipy` ?).
- `numpy` **écrase** des fonctions natives : `sum`, `max`, `min`, `abs` → comportements différents → bugs silencieux.
- Code impossible à relire pour un tiers.

Règle d'or : `import *` est toléré **uniquement** en session interactive de débogage, jamais dans un TP ou un projet.

2.4 Le bloc d'import standard du cours

En tête de **chaque notebook de TP**, tu écriras exactement ce bloc — mémorise-le :

```
# --- Bibliotheques standard ---
import os
import sys

# --- Calcul scientifique ---
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
from scipy.stats import norm, probplot
from scipy.optimize import curve_fit

# --- Affichage des figures dans le notebook ---
%matplotlib inline
```

Listing 1 – Bloc d'import standard à mémoriser.

Rappels utiles

- **Ordre** : standard → tiers → locaux (convention PEP8).
- `%matplotlib inline` est une **commande magique Jupyter** (préfixe `%`) qui affiche les figures directement dans le notebook.
- `os` et `sys` servent dès qu'on manipule des fichiers (`os.getcwd()`, `os.listdir()`).

2.5 Comment ça se passe dans un notebook

Une seule cellule d'import, en haut. Une fois la cellule exécutée, les bibliothèques restent chargées **pour toutes les cellules suivantes**. Pas besoin de réimporter à chaque cellule.

Conséquence pratique : si tu redémarres le noyau (**Kernel → Restart**), tu perds tous les imports. Il faut **réexécuter la cellule d'import**. C'est la cause n°1 des `NameError: name 'np' is not defined` chez les débutants.

Piège classique

En cas d'erreur bizarre, ne cherche pas : **Kernel → Restart & Run All**. Tu repars d'un état propre.

2.6 Erreurs classiques et leur correction

Message	Cause	Solution
`ModuleNotFoundError: No module named 'numpy'`	Bibliothèque non installée	`pip install numpy` (ou réinstaller Anaconda)
`NameError: name 'np' is not defined`	Import oublié ou noyau redémarré	Réexécuter la cellule d'import
`ImportError: cannot import name 'X' from 'Y'`	Faute de frappe ou mauvaise version	Vérifier l'orthographe, consulter la doc
`ImportError: attempted relative import`	Lancement direct d'un fichier de package	Utiliser `python -m package.module`

2.7 `pip install` vs `import`

Commande	Quand	Rôle
`pip install numpy`	**Une fois**, dans le terminal	Télécharge et installe sur le disque
`import numpy as np`	**À chaque** script/notebook	Charge en mémoire pour l'utiliser

Analogie

`pip install` = acheter le livre et le ranger dans ta bibliothèque. `import` = sortir le livre de l'étagère pour le lire. Une fois installé, tu n'as plus jamais besoin de refaire `pip install`. Mais tu refais `import` à chaque notebook.

2.8 Bonnes pratiques à adopter dès maintenant

1. **Toujours** en haut du notebook, dans une cellule dédiée.
2. **Toujours** les mêmes alias : `np`, `pd`, `plt`. Ne renomme jamais.
3. **Jamais** `import *`.
4. **Regroupe** les imports par catégorie avec des commentaires.
5. **Importe le minimum** : `from scipy.optimize import curve_fit`, pas tout `scipy.optimize`.
6. **En cas de doute** sur l'origine d'un nom : cherche le `import` correspondant en haut du fichier.

2.9 Exercices

Exercice 1 — Écrire ton bloc d'import

Crée une nouvelle cellule dans un notebook vierge et écris le bloc d'import standard du cours (section 2.5). Exécute-la : aucun message ne doit apparaître. C'est normal — un import silencieux est un import réussi.

Exercice 2 — Vérifier les versions

```
import numpy as np, pandas as pd, matplotlib, scipy
print(f"NumPy      : {np.__version__}")
print(f"Pandas     : {pd.__version__}")
print(f"Matplotlib : {matplotlib.__version__}")
print(f"SciPy      : {scipy.__version__}")
```

Note les quatre numéros. Si l'un d'eux est très ancien (NumPy < 1.20 par exemple), mets à jour : `pip install -upgrade numpy`.

Exercice 3 — Diagnostiquer l'erreur

Dans une cellule isolée, écris :

```
x = np.array([1, 2, 3])
print(x.mean())
```

Sans exécuter la cellule d'import au préalable. Observe l'erreur exacte. Puis ajoute `import numpy as np` et réexécute. C'est le scénario classique du débutant — tu viens de le vivre pour ne plus le reproduire.

Exercice 4 — Comprendre l'alias

Écris deux versions d'un même calcul et vérifie que le résultat est identique :

```
import numpy
x1 = numpy.sqrt(numpy.array([4, 9, 16]))

import numpy as np
x2 = np.sqrt(np.array([4, 9, 16]))

assert (x1 == x2).all()
print("Identique :", x1)
```

L'alias n'est qu'un surnom. Le résultat est strictement le même.

Exercice 5 — Lire une erreur d'import

Lance volontairement :

```
from scipy.stats import normall
```

Lis le message d'erreur. Identifie quelle lettre en trop cause le problème. Corrige.

3 Variables, types et opérations

Objectifs de la section

Manipuler les variables, comprendre les types de base, écrire des expressions arithmétiques et formater proprement l'affichage.

3.1 Variables et types

```
L = 1.000           # un float (nombre a virgule) : longueur en m
N = 50              # un int (entier) : nombre de mesures
nom = "pendule"     # une str (chaine de caracteres)
flag = True         # un bool (vrai/faux)

print(L, N, nom, flag)
print(type(L))      # <class 'float'>
```

3.2 Opérations arithmétiques

```
g = 9.81
T = 2 * 3.14159 * (L / g) ** 0.5    # ** = puissance
print(f"T = {T:.4f} s")              # f-string : formatage
```

Le préfixe `f` devant les guillemets permet d'insérer des variables dans le texte. `:.4f` = 4 chiffres après la virgule.

3.3 Les f-strings, ton meilleur ami

```
R, uR = 100.0, 1.0
print(f"R = {R} +/- {uR} ohm")
print(f"R = {R:.1f} +/- {uR:.2f} ohm")
print(f"R = {R:8.2f}")      # largeur 8, 2 decimales, aligne
```

Symbole ±

Dans le code ci-dessus, on utilise `+/-` au lieu de `±` pour éviter tout problème d'encodage. Python 3 supporte parfaitement `±` si ton fichier est en UTF-8 — tu peux l'utiliser dans tes propres scripts.

Exercice 6 — Calculer une période

Calcule la période d'un pendule pour `L = 0.5`, `L = 1.0`, `L = 2.0`, et affiche les trois résultats avec 4 décimales. Utilise `g = 9.81`.

4 Listes et structures de contrôle

Objectifs de la section

Savoir créer et parcourir une liste, écrire une boucle `for`, une condition `if/else`.

4.1 Listes

```
mesures = [2.01, 1.99, 2.00, 2.02, 1.98]
print(len(mesures))      # 5
print(mesures[0])        # 2.01 (le premier, indice 0)
print(mesures[-1])       # 1.98 (le dernier)
```

4.2 Boucles `for`

```
for i in range(5):
    print(i, mesures[i])
```

4.3 Conditions `if / else`

```
z = 1.7
if abs(z) < 2:
    print("compatible")
else:
    print("incompatible")
```

Exercice 7 — Compter des valeurs

Écris une boucle qui parcourt `mesures` et compte combien de valeurs sont supérieures à 2.00.

Piège classique

Les listes Python **ne sont pas faites pour le calcul scientifique**. Pour la suite, on passe à NumPy.

5 NumPy — le cœur du calcul scientifique

Objectifs de la section

Savoir créer et manipuler des tableaux NumPy, utiliser la vectorisation et les statistiques de base (`mean`, `std`, `sum`).

5.1 Créer un tableau

```python
import numpy as np

x = np.array([1.0, 2.0, 3.0, 4.0])
print(x.shape)       # (4,)
print(x.dtype)       # float64

np.zeros(5)                    # [0. 0. 0. 0. 0.]
np.ones(3)                     # [1. 1. 1.]
np.arange(0, 10, 2)            # [0 2 4 6 8]
np.linspace(0, 1, 5)           # [0. 0.25 0.5 0.75 1.]
```

5.2 Opérations vectorisées (le super-pouvoir)

```python
x = np.array([1, 2, 3, 4])
y = x * 2           # [2 4 6 8]
z = x ** 2          # [1 4 9 16]
w = x + y           # [3 6 9 12]
print(x.mean())     # 2.5
print(x.std(ddof=1))   # ecart-type experimental
print(x.sum())      # 10
```

Rappels utiles

`ddof=1` = écart-type corrigé (dénominateur $N-1$), celui qu'on utilise en physique pour des mesures répétées.

Exercice 8 — Cœur du TP 1

```python
T = np.array([2.01, 1.99, 2.00, 2.02, 1.98, 2.03, 1.97, 2.01, 1.99, 2.00])
```

Calcule : N (taille), la moyenne, l'écart-type (`ddof=1`), l'erreur-type $= s/\sqrt{N}$. Affiche chaque résultat avec f-string.

Exercice 9 — Cœur du TP 2

```python
R, uR = 100.0, 1.0
I, uI = 0.050, 0.001
```

Calcule $U = R\,I$ et

$$u_U = \sqrt{(I\,u_R)^2 + (R\,u_I)^2}.$$

Affiche au format : `U = 5.000 +/- 0.112 V`. **Indice** : `np.sqrt(...)`.

6 Matplotlib — visualiser

Objectifs de la section

Tracer une courbe, un histogramme, une barre d'erreur, et organiser plusieurs figures côte à côte.

6.1 Figure minimale

```
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
plt.figure(figsize=(6, 4))
plt.plot(x, np.sin(x), "b-", label="sin(x)")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("Ma premiere figure")
plt.legend()
plt.grid(True)
plt.show()
```

6.2 Histogramme

```
T = np.random.normal(2.006, 0.015, 50)   # donnees simulees
plt.hist(T, bins=10, edgecolor="k", alpha=0.7)
plt.xlabel("T (s)")
plt.ylabel("Effectif")
plt.show()
```

6.3 Barres d'erreur (crucial pour les TP)

```
I = np.array([0.01, 0.02, 0.03, 0.04, 0.05])
U = np.array([1.02, 1.98, 3.01, 4.03, 4.99])
uU = np.full_like(U, 0.02)     # incertitude constante

plt.errorbar(I, U, yerr=uU, fmt="o", capsize=3)
plt.xlabel("I (A)")
plt.ylabel("U (V)")
plt.show()
```

6.4 Sous-figures (pour TP 1)

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].hist(T, bins=10)
ax[0].set_title("Histogramme")
ax[1].plot(x, np.sin(x))
ax[1].set_title("Sinus")
plt.tight_layout()
plt.show()
```

Exercice 10 — Comparer deux planètes

Trace sur la même figure deux courbes de période de pendule $T(L)$: pour `L` allant de 0.1 à 2.0 m, avec `g = 9.81` (Terre) puis `g = 1.62` (Lune). Ajoute une légende et des axes nommés.

7 Pandas — lire des fichiers de données

Objectifs de la section

Lire un fichier CSV, explorer un DataFrame, extraire une colonne en tableau NumPy, écrire un CSV.

7.1 DataFrame et CSV

```python
import pandas as pd

# Lire un CSV
df = pd.read_csv("mesures_pendule.csv")
print(df.head())          # 5 premieres lignes
print(df.shape)           # (lignes, colonnes)
print(df.columns)         # noms des colonnes
print(df.describe())      # stats rapides
```

7.2 Accéder à une colonne

```python
T = df["T_s"].to_numpy()     # conversion en array NumPy
# ou directement :
print(df["T_s"].mean())
```

7.3 Écrire un CSV

```python
pd.DataFrame({"essai": [1, 2, 3], "T_s": [2.01, 1.99, 2.00]}) \
  .to_csv("test.csv", index=False)
```

Exercice 11 — Générer puis relire un CSV

1. Génère un CSV avec 30 mesures de période autour de 2.006 s (bruit 15 ms) via NumPy et `pd.DataFrame`.
2. Lis-le avec `pd.read_csv`.
3. Affiche moyenne, écart-type et erreur-type.

Indice : `np.random.normal(mu, sigma, N)`.

8 Nombres aléatoires

Objectifs de la section

Générer du bruit gaussien reproductible et réaliser une simulation Monte Carlo simple.

```python
rng = np.random.default_rng(42)     # 42 = graine, reproductibilite

# 5 tirages gaussiens
tirage = rng.normal(loc=0, scale=1, size=5)

# Monte Carlo : 100 000 tirages
Rmc = rng.normal(100.0, 1.0, 100_000)
Imc = rng.normal(0.050, 0.001, 100_000)
Umc = Rmc * Imc
print(f"U_MC = {Umc.mean():.4f} +/- {Umc.std(ddof=1):.4f} V")
```

Rappels utiles

Pourquoi `default_rng(graine)` ? En fixant la graine (ici 42), on obtient **toujours les mêmes** nombres aléatoires. C'est indispensable pour que ton TP soit reproductible par le correcteur.

Exercice 12 — Simuler une série de mesures

Simule 50 mesures de période autour de 2.006 s avec un bruit de 15 ms, puis calcule moyenne, écart-type et erreur-type. Compare avec `T_th = 2.006 s`.

9 SciPy — distributions et ajustements

Objectifs de la section

Utiliser `norm` pour ajuster une gaussienne, tracer un QQ-plot, et réaliser un ajustement de modèle avec `curve_fit`.

9.1 Loi normale et ajustement

```python
from scipy.stats import norm

# Densite de probabilite
x = np.linspace(1.9, 2.1, 100)
y = norm.pdf(x, loc=2.006, scale=0.015)
plt.plot(x, y)

# Ajuster une gaussienne a des donnees
mu, sigma = norm.fit(T)
print(f"mu = {mu:.4f}, sigma = {sigma*1e3:.2f} ms")
```

9.2 QQ-plot (test de normalité, utilisé au TP 1)

```python
from scipy.stats import probplot
probplot(T, dist="norm", plot=plt)
plt.show()
```

9.3 Ajustement linéaire pondéré (aperçu TP 3)

```python
from scipy.optimize import curve_fit

def modele(I, R, b):
    return R * I + b

popt, pcov = curve_fit(modele, I_data, U_data,
                       sigma=u_U, absolute_sigma=True)
R_fit, b_fit = popt
uR, ub = np.sqrt(np.diag(pcov))
print(f"R = {R_fit:.2f} +/- {uR:.2f} ohm")
```

Exercice 13 — Ajuster une gaussienne

Simule 50 mesures `T` (moyenne 2.006, écart-type 0.015), ajuste une gaussienne, compare `mu` et `sigma` aux valeurs "vraies", trace l'histogramme avec la gaussienne superposée.

10 Assemblage — tu es prêt pour les TP

Objectifs de la section

Exécuter deux scripts complets, l'un correspondant au TP 1, l'autre au TP 2, sans aucune erreur.

10.1 Squelette complet du TP 1

Recopie-le, exécute-le, vérifie que la figure s'affiche.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, probplot

# 1. Generation / chargement
rng = np.random.default_rng(7)
T_true = 2 * np.pi * np.sqrt(1.000 / 9.81)
T = T_true + rng.normal(0, 0.015, size=50)
# (ou : T = pd.read_csv("mesures_pendule.csv")["T_s"].to_numpy())

# 2. Exploration
N   = T.size
Tb  = T.mean()
s   = T.std(ddof=1)
sem = s / np.sqrt(N)
print(f"N = {N}, <T> = {Tb:.4f} s, "
      f"s = {s*1e3:.2f} ms, sem = {sem*1e3:.2f} ms")

# 3. Ajustement gaussien
mu, sigma = norm.fit(T)
print(f"Ajustement : mu = {mu:.4f} s, sigma = {sigma*1e3:.2f} ms")

# 4. Comparaison a la theorie
z = (Tb - T_true) / sem
print(f"T_th = {T_true:.4f} s, z = {z:.2f}")

# 5. Figures
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].hist(T, bins=10, density=True, alpha=0.6,
           edgecolor="k", label="donnees")
x = np.linspace(T.min() - 0.05, T.max() + 0.05, 200)
ax[0].plot(x, norm.pdf(x, mu, sigma), "r-",
           label=f"Gauss mu={mu:.4f}, sigma={sigma*1e3:.1f} ms")
ax[0].set_xlabel("T (s)")
ax[0].legend()
probplot(T, dist="norm", plot=ax[1])
plt.tight_layout()
plt.savefig("fig_tp1.png", dpi=150)
plt.show()
```

Listing 2 – Script complet TP 1 (mesures répétées).

10.2 Cœur du TP 2 (propagation et Monte Carlo)

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 1. Donnees
R, uR = 100.0, 1.0      # ohm
```

```python
I, uI = 0.050, 0.001    # ampere

# 2. Propagation analytique
U  = R * I
uU = np.sqrt((I * uR)**2 + (R * uI)**2)
print(f"U = {U:.4f} +/- {uU:.4f} V")

# 3. Contributions
cR = (I * uR)**2 / uU**2
cI = (R * uI)**2 / uU**2
print(f"R contribue {cR*100:.1f} %, I contribue {cI*100:.1f} %")

# 4. Monte Carlo
rng = np.random.default_rng(0)
Umc = rng.normal(R, uR, 100_000) * rng.normal(I, uI, 100_000)
print(f"MC : U = {Umc.mean():.4f} +/- {Umc.std(ddof=1):.4f} V")

# 5. Figure
plt.hist(Umc, bins=80, density=True, alpha=0.6,
         edgecolor="none", label="MC")
x = np.linspace(Umc.min(), Umc.max(), 300)
plt.plot(x, norm.pdf(x, U, uU), "r-", label="Gauss")
plt.xlabel("U (V)")
plt.legend()
plt.savefig("fig_tp2.png", dpi=150)
plt.show()
```

Listing 3 – Script complet TP 2 (loi d'Ohm).

Si les deux blocs tournent et produisent des figures, **tu sais faire les TP 1 et 2**.

11 Checklist avant le TP

Coche chaque case. Si une case reste vide, retourne à la section correspondante.

☐ Je sais écrire une f-string avec `{x:.4f}`.

☐ Je sais choisir entre `import module`, `import module as alias`, `from module import fonction`.

☐ Je sais pourquoi `from x import *` est une mauvaise pratique.

☐ Je connais le bloc d'import standard du cours par cœur.

☐ Je sais créer un `np.array` et faire `mean()`, `std(ddof=1)`, `sqrt`.

☐ Je sais générer `np.random.normal(mu, sigma, N)` avec une graine fixée.

☐ Je sais lire un CSV avec `pd.read_csv` et extraire une colonne en array.

☐ Je sais tracer un histogramme et une barre d'erreur avec Matplotlib.

☐ Je sais superposer une gaussienne à un histogramme avec `norm.fit` + `norm.pdf`.

☐ Je sais calculer une incertitude composée `sqrt((I*uR)**2 + (R*uI)**2)`.

☐ Je sais faire un Monte Carlo à 100 000 tirages et en extraire moyenne ± écart-type.

☐ Je comprends pourquoi on fixe la graine (`default_rng(42)`).

☐ J'ai exécuté le squelette du TP 1 et du TP 2 sans erreur.

12 Dépannage — les blocages les plus fréquents

Symptôme	Cause probable	Solution
`ModuleNotFoundError`	Bibliothèque non installée	`pip install numpy` (ou réinstaller Anaconda)
`NameError: name 'np' is not defined`	Import oublié ou noyau redémarré	Réexécuter la cellule d'import
`FileNotFoundError: mesures_pendule.csv`	Fichier dans un autre dossier	Vérifier `import os; os.getcwd()`
Figure vide	Oubli de `plt.show()` ou `plt.savefig`	Ajouter `plt.show()`
Valeurs aberrantes	Oubli de `ddof=1` ou unités	Vérifier unités (mA vs A) et `ddof=1`
`zsh: command not found: python`	Environnement non activé	Ouvrir un terminal Anaconda

Stratégie de débogage

En dernier recours : **copie ton message d'erreur complet** dans un moteur de recherche. 90 % des erreurs de débutant ont déjà une réponse sur Stack Overflow ou la documentation officielle.

13 Pour aller plus loin (optionnel)

Une fois les TP réussis, trois pistes utiles :

1. **Broadcasting NumPy** — opérations entre tableaux de formes différentes.
2. **`pandas` groupby** — regrouper des mesures par condition (utile au TP 7).
3. **`scipy.optimize.curve_fit` en profondeur** — ajustements pondérés, modèles non linéaires (TP 3–4).

— Fin du tutoriel —
Bonne pratique et bon TP.